



Tema 2: Ordenamiento

Estructura de Datos

Universidad Nacional de la Patagonia Austral
Unidad Académica Río Gallegos

Indice

- Analisis de algoritmos: ordenes de complejidad
- Introducción:
 - Concepto
 - Objetivos
 - Criterios de clasificación
- Algoritmos básicos
 - Inserción
 - Selección
 - Intercambio
- Algoritmos avanzados
 - Introducción
 - Mergesort, heapsort, quicksort, shellsort, shakesort.
- Otros algoritmos avanzados
 - Introducción
 - Binsort
 - Radixsort



Introducción

- Dada una colección que contiene un número de elementos, el ordenamiento es el proceso de reacomodar un conjunto de elementos dados en un orden específico.
- Objetivo: facilitar la posterior búsqueda de elementos dentro del conjunto ordenado.
- Se toma como estructura de referencia, el arreglo y números enteros.
- Las búsquedas mas eficientes se realizan en conjuntos de datos ordenados.



Objetivos

- Analizar el desempeño de diferentes algoritmos empleados en el ordenamiento de números o cadenas de caracteres.
- Determinar que algoritmo es más eficiente para un problema particular dado.

Generalidades

- Dado un conjunto de n elementos $a_1, a_2, \dots, a_n$ y una relación de orden total ($\leq$) sobre ellos, el problema de la ordenación consiste en encontrar una permutación de esos elementos, ordenada de forma creciente.
- Para la entrada 2-5-4-1-3; la permutación que produce el ordenamiento es:
 - $p(1)=\{2\}$, $p(2)=\{2,5\}$, $p(3)=\{2,4,5\}$, $p(4)=\{1,2,4,5\}$, y $p(5)=\{1,2,3,4,5\}$.
 - Así: $1 \leq 2 \leq 3 \leq 4 \leq 5$
- Para una secuencia de n elementos, el número de permutaciones considerando n datos elementos es $n!$

Clasificación

- Ordenamiento comparativos
 - Ordenan comparando
 - Todos los vistos hasta ahora
 - Selección, inserción, burbuja...
- Ordenamiento no comparativos
 - Ordenan usando información adicional
 - No se puede aplicar siempre



Criterios de clasificación

- Eficiencia:
 - $O(n^2)$: Burbuja, Inserción, Selección, ShakerSort.
 - $O(n \log n)$: MergeSort, HeapSort, QuickSort.
 - Otros: ShellSort $O(n^{1.25})$, BinSort $O(n)$, RadixSort $O(n)$.
- Tipo de ordenación:
 - Ordenaciones por intercambio: Burbuja, ShakerSort, QuickSort.
 - Ordenaciones por selección: Selección, HeapSort.
 - Ordenaciones por inserción: Inserción, ShellSort.
 - Ordenaciones por distribución: BinSort, RadixSort.
 - Ordenaciones por intercalación: MergeSort.
- Puntos de análisis a tener en cuenta:
 - Complejidad
 - Peor caso, mejor caso y caso medio



Algoritmos básicos de ordenación

Inserción, Selección e Intercambio

Ordenación por Inserción

- El ordenamiento por inserción es una manera muy natural de ordenar para un ser humano.
- Similar a método usado por un jugador para ordenar un mazo de cartas.
- Este método toma cada elemento del arreglo para ser ordenado y lo compara con los que se encuentran en posiciones anteriores a las de él, dentro del arreglo.
- Se analiza a partir del 2do elemento.
- El método de Inserción realiza $n-1$ iteraciones sobre el arreglo, dejando en la i -ésima etapa ($2 \leq i \leq n$) ordenado el arreglo.
- La forma de hacerlo es colocando en cada iteración el elemento en su sitio correcto, aprovechando el hecho de que el arreglo ya ha sido previamente ordenado.

Ordenación por Inserción

- Este método puede ser implementado de forma iterativa como sigue:

```
public static void insertionSort(int[] vec) {
    for (int i=1; i < vec.length; i++) {
        int aux=vec[i];
        int j=i-1;
        while(j>=0 && vec[j]>aux) {
            vec[j+1]=vec[j];
            j-=1;
        }
        vec[j+1]=aux;
    }
}
```

Ordenación por Inserción

- Ejemplo:
- Array original:
 - 44 55 12 42 94 18 06 67
- Proceso:
 - 44 (55) 12 42 94 18 06 67
 - 44 55 (12) 42 94 18 06 67
 - 12 44 55 (42) 94 18 06 67
 - 12 42 44 55 (94) 18 06 67
 - 12 42 44 55 94 (18) 06 67
 - 12 18 42 44 55 94 (06) 67
 - 06 12 18 42 44 55 94 (67)
 - 06 12 18 42 44 55 67 94

Ord. Inserción

Análisis de complejidad:

- Se evalúa método `insertionSort(vec)`.
- En el mejor caso:
 - el bucle interno no se realiza nunca, y por lo tanto el orden de magnitud es $O(n)$
- En el peor caso:
 - hay que llevar cada elemento hasta su posición final, con lo que el bucle interno se realiza siempre de $i-1$ veces. Así, en este caso es $O(n^2)$.
- En el caso medio, la probabilidad de que el elemento se sitúe en alguna posición k de las i primeras será de $1/i$. El número de veces que se repetirá el bucle `while` en este caso es $(i-k)$, con lo cual el número medio de operaciones que se realizan en el bucle es $O(n^2)$.

```
public static void insertionSort(int[] vec) {
    for (int i=1; i < vec.length; i++) {
        int aux=vec[i];
        int j=i-1;
        while (j>=0 && vec[j]>aux) {
            vec[j+1]=vec[j];
            j--;
        }
        vec[j+1]=aux;
    }
}
```

Ordenación por Inserción

- Conclusiones:
 - Los ordenes de complejidad difieren.
 - Mejor caso
 - una lista ordenada: $O(n)$ por iteración lineal.
 - Peor caso
 - una lista ordenada inversamente: $O(n^2)$.
 - Caso medio:
 - Cuadrática $O(n^2)$.
- Se recomienda:
 - Para arreglos casi ordenados.
- Ventajas:
 - Fácil implementación.
 - Requerimientos mínimos de memoria.
- Desventajas:
 - Lento.
 - Realiza numerosas comparaciones.

Ordenación por Selección

- Este algoritmo también es sencillo. Consiste en lo siguiente:
 - Buscar el elemento más pequeño de la lista.
 - Se intercambia con el elemento ubicado en la primera posición de la lista.
 - Buscar el segundo elemento más pequeño de la lista.
 - Se intercambia con el elemento que ocupa la segunda posición en la lista.
 - Repetir este proceso hasta que se haya ordenado toda la lista.



Ordenación por Selección

- Este método puede ser implementado como sigue:

```
public static void selectionSort(int[] vec) {
    for(int i=0; i<vec.length - 1; i++) {
        int pos=i;
        for(int j=i + 1; j<vec.length; j++)
            if(vec[j]<vec[pos])
                pos=j;
        interChange(vec, pos, i);
    }
}
```



Ordenación por Selección

- Ejemplo:

- 25 3 12 19 2 ① 9 6
- 1 3 12 19 ② 25 9 6
- 1 2 12 19 ③ 25 9 6
- 1 2 3 19 12 25 9 ⑥
- 1 2 3 6 12 25 ⑨ 19
- 1 2 3 6 9 25 ⑫ 19
- 1 2 3 6 9 12 25 ⑮
- 1 2 3 6 9 12 19 25

Ordenación por Selección

Analisis de complejidad:

- El ciclo externo se ejecuta n veces para una lista de n elementos.
- Cada búsqueda requiere comparar todos los elementos no clasificados. La complejidad es de $O(n^2)$.
- Este algoritmo presenta un comportamiento constante independiente del orden de los datos. Siendo la complejidad promedio de $O(n^2)$.
- En resumen:
 - En el mejor caso:
 - Orden de magnitud es de $O(n^2)$
 - En el peor caso:
 - Orden de magnitud es de $O(n^2)$
 - En el caso medio:
 - Orden de magnitud es de $O(n^2)$

Ordenación por Selección

- **Ventajas:**
 - Fácil implementación.
 - No requiere memoria adicional.
 - Realiza pocos intercambios.
 - Rendimiento constante: O magnitud iguales para todos los casos.
- **Desventajas:**
 - Lento.
 - Realiza numerosas comparaciones.
 - Es ineficiente para conjuntos grandes de elementos.

Ordenación por Selección

- Conclusiones:
 - El algoritmo es de complejidad cuadrática $O(n^2)$.
 - Método recomendado para ordenar pocos registros de gran tamaño.

Ordenación por intercambio

- Este método de ordenación también es conocido como **Método de la Burbuja**.
- **Método:**
 - Consiste en ordenar una lista de elementos realizando comparación de pares adyacentes.
 - Cada comparación pone al elemento mas pequeño no ordenado en su lugar correcto, haciendo cambios en la posición de los demás elementos.
 - La forma de llevar a cabo esto es mediante **sucesivos intercambios**, que en cierto modo van a ir "**promocionando**" (**burbuja**) un determinado valor desde el final del vector hasta la posición i -ésima
 - La 1ra. iteración ubica al valor mas grande en la ultima posición.
 - De esta forma el elemento menor va "burbujeando" hasta el principio y así sucesivamente.

Ordenación por Intercambio

- Este método puede ser implementado como sigue:

```
public static void bubbleSort(int[] vec) {
    for(int i=vec.length - 1; i>=1; i--)
        for(int j=0; j<=i - 1; j++)
            if(vec[j]>vec[j + 1])
                interChange(vec, j, j + 1);
}
```

Ordenación por Intercambio

Ejemplo:

- Array original:
 - 36 24 10 6 12
- 1º iteración n-1:
 - 24 10 6 12 36
- 2º iteración n-2:
 - 10 6 12 24 36
- 3º iteración n-3:
 - 6 10 12 24 36
- 4º iteración n-4:
 - 6 10 12 24 36



Ordenación por Intercambio

Análisis de complejidad:

- El ciclo interno se ejecuta n veces para una lista de n elementos. El ciclo externo también se ejecuta n veces.
- Es decir, la complejidad es $n * n = O(n^2)$.
- El comportamiento del caso promedio depende del orden de entrada de los datos, pero es sólo un poco mejor que el del peor caso, y sigue siendo $O(n^2)$.



Ordenación por Intercambio

- Por lo tanto, el algoritmo es de complejidad cuadrática $O(n^2)$.
- Este algoritmo funciona de forma parecida al de Selección, pero haciendo más trabajo para llevar cada elemento a su posición.
- Es el peor de los tres vistos hasta ahora, no sólo en cuanto al tiempo de ejecución, sino también respecto al número de comparaciones y de intercambios que realiza.
- Una posible mejora que puede admitir este algoritmo es el control de la existencia de una pasada sin intercambios; en ese momento el arreglo estará ordenado.



Ordenación por Intercambio

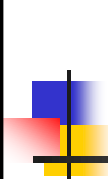
- **Burbuja mejorado:**
- Puede ser que los datos queden ordenados antes de completar el ciclo externo.
- Se puede modificar el algoritmo para que verifique si se han realizado intercambios. Si no se han hecho entonces se termina con la ejecución, pues eso significa que los datos ya están ordenados.
- Algoritmo:

```
public static void bubbleImprovedSort(int[] vec) {  
    int i=vec.length - 1;  
    boolean ordered;  
    do {  
        ordered=true;  
        for(int j=0; j<=i - 1; j++)  
            if(vec[j]>vec[j+1]) {  
                interChange(vec, j, j + 1);  
                ordered=false;  
            }  
        i--;  
    }  
    while(i>=1 & !ordered);  
}
```

Estructura de Datos

Ordenamiento

24



Algoritmos avanzados de ordenación

MergeSort, HeapSort, QuickSort,
ShellSort y ShakeSort



Introducción

- Los algoritmos de ordenación interna: intercambio, selección e inserción, son fáciles de entender y de codificar, sin embargo poco eficientes y no recomendables para ordenar listas de muchos elementos.
- La complejidad (Orden Magnitud) es cuadrática: $O(n^2)$.
- Los métodos básicos pueden describirse en términos de comparaciones e intercambios.
- Los algoritmos avanzados usan otra técnica mas elaborada, como divide y vencerás, para reducir el tiempo de ordenamiento.



Mergesort

- Desarrollado en 1945 por John Von Neumann.
- Utiliza la técnica de "Divide y Vencerás".
- Estrategia:
 - consiste en dividir el arreglo en dos arreglos,
 - ordenarlos mediante llamadas recursivas.
 - Finalmente combinar los dos arreglos ya ordenados.
- El ordenamiento por mezcla incorpora dos ideas principales para mejorar su tiempo de ejecución:
 - Una lista pequeña necesitará menos pasos para ordenarse que una lista grande.
 - Se necesitan menos pasos para construir una lista ordenada a partir de dos listas también ordenadas, que a partir de dos listas desordenadas.

Mergesort (pasos)

Ordenar una secuencia S de elementos:

- Si S tiene uno o ningún elemento, está ordenada
- Si S tiene al menos dos elementos se divide en dos secuencias S1 y S2, S1 conteniendo los primeros $n/2$, y S2 los restantes.
- Ordenar S1 y S2, aplicando recursivamente este procedimiento.
- Mezclar S1 y S2 ordenadamente en S

Mezclar dos secuencias ordenadas S1 y S2 en S:

- Se tienen referencias al principio de cada una de las secuencias a mezclar (S1 y S2).
- Mientras en alguna secuencia queden elementos, se inserta en la secuencia resultante (S) el menor de los elementos referenciados y se avanza esa referencia una posición.

Mergesort

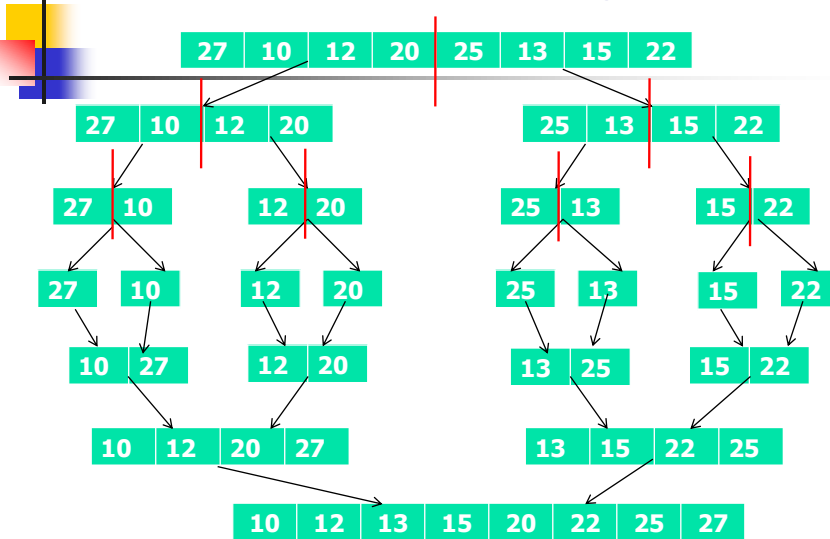
■ Implementación:

```
public static void mergeSort(int[] vec) {
    int[] tmp=new int[vec.length];
    mergeSort(vec, tmp, 0, vec.length-1);
}

private static void mergeSort(int[] vec, int[]
tmp, int left, int right) {
    if(left<right) {
        int center=(right + left) / 2;
        mergeSort(vec, tmp, left, center);
        mergeSort(vec, tmp, center+1, right);
        merge(vec, tmp, left, center, right);
    }
}
```

```
private static void merge(int[] vec, int[] tmp,
int left, int center, int right) {
    int aptr=left;
    int bptr=center+1;
    int cptr=left;
    while (aptr<=center & bptr<=right) {
        if (vec[aptr]<vec[bptr])
            tmp[cptr++]=vec[aptr++];
        else
            tmp[cptr++]=vec[bptr++];
    }
    while (aptr<=center)
        tmp[cptr++]=vec[aptr++];
    while (bptr<=right)
        tmp[cptr++]=vec[bptr++];
    for(int i=left; i<=right; i++)
        vec[i]=tmp[i];
}
```

MERGESORT: ejemplo



Estructura de Datos

Ordenamiento

30

Mergesort

Análisis de complejidad:

- Se comporta siempre de la misma manera.
- Para ordenar un arreglo de tamaño n se ordenan 2 arreglos de tamaño $n/2$, de aquí el $2T(n/2)$, y luego se consume $O(n)$ en realizar la mezcla. Resolviendo la ecuación recurrente tenemos que **$T(n) = O(n \log n)$** , en cualquiera de los casos (mejor, peor o medio).
- La mayoría de los algoritmos de ordenamiento recursivo tiene complejidad logarítmica: $O(n \log n)$.
- La eficiencia de este algoritmo es bastante notable en tiempo de ejecución en comparación con otros, ya que su manera de trabajo por grupos pequeños agiliza la organización de los datos.

Estructura de Datos

Ordenamiento

31



Heapsort

- Consiste en almacenar todos los elementos del vector a ordenar en un montículo (heap), y luego extraer el nodo que queda como nodo raíz del montículo (cima) en sucesivas iteraciones obteniendo el conjunto ordenado.
- Es un árbol binario completo.
 - Basa su funcionamiento en una propiedad de los montículos, por la cual, la cima contiene siempre el mayor elemento de todos los almacenados en él.
- Debido a esta propiedad de orden, el valor máximo está en el nodo raíz.



Heapsort

- Procedimiento:
 - Convertir el arreglo en un montículo.
 - Construir un arreglo ordenado de atrás hacia adelante (mayor a menor) repitiendo los siguientes pasos:
 - Sacar el valor máximo en el montículo (posición 1)
 - Colocar ese valor en el arreglo ordenado.
 - Reconstruir el montículo con un elemento menos.
 - Utilizar el mismo arreglo para el montículo y el arreglo ordenado.
- A continuación se muestra el algoritmo de este método.

Heapsort

```
public static void heapSort(int[] vec) {
    int index;
    for(index=vec.length - 1; index>=0; index--)
        reHeapDown(vec, index, vec.length);
    for(index=vec.length - 1; index>0; index--) {
        interChange(vec, 0, index);
        reHeapDown(vec, 0, index);
    }
}
```

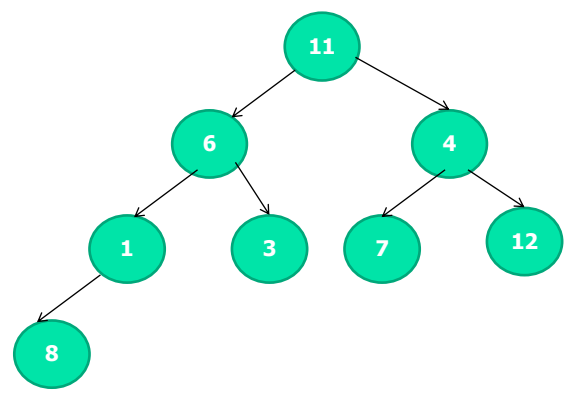
Método que construye un montículo a partir del arreglo

```
private static void reHeapDown(int vec[], int length, int index) {
    boolean done=false;
    int aux=vec[length];
    int parent=length;
    int child=2 * (length + 1) - 1;
    while(child<index & !done) {
        if(vec[child]<vec[child + 1])
            child++;
        if(aux>vec[child])
            done=true;
        else {
            vec[parent]=vec[child];
            parent=child;
            child=2 * (parent + 1) - 1;
        }
    }
    vec[parent]=aux;
}
```

Método que "empuja" un elemento hasta su posición definitiva en el montículo, reconstruyendo la estructura de montículo en el arreglo

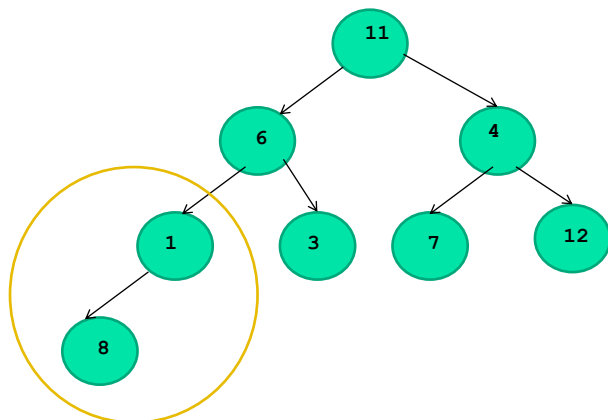


HEAPSORT: ejemplo





HEAPSORT: conversión a Heap



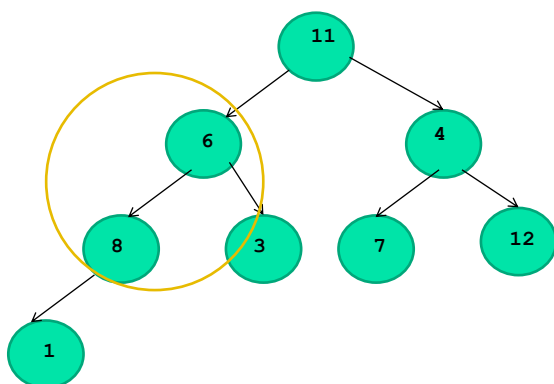
Estructura de Datos

Ordenamiento

36



HEAPSORT: conversión a Heap



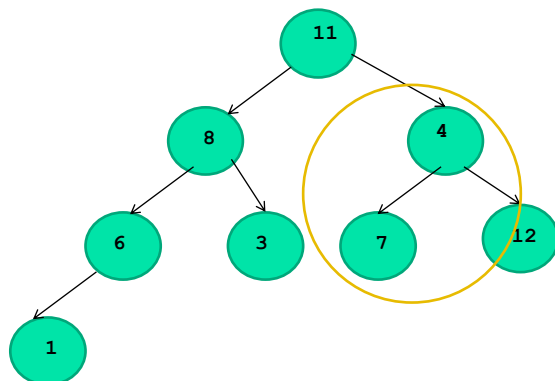
Estructura de Datos

Ordenamiento

37



HEAPSORT: conversión a Heap



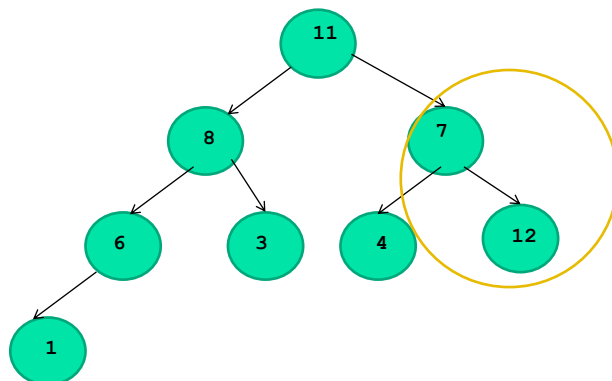
Estructura de Datos

Ordenamiento

38



HEAPSORT: conversión a Heap



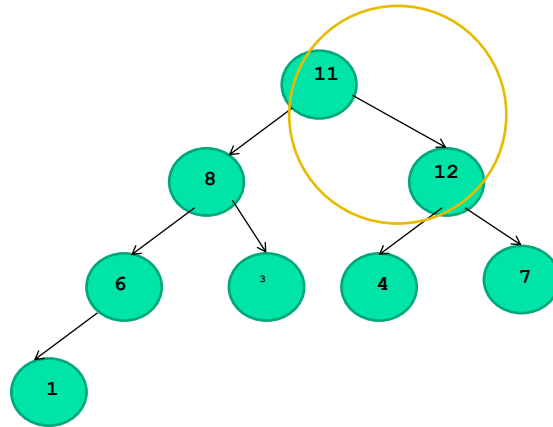
Estructura de Datos

Ordenamiento

39



HEAPSORT: conversión a Heap



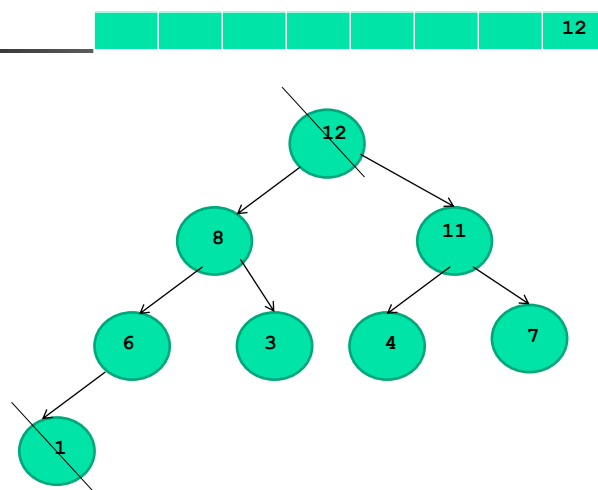
Estructura de Datos

Ordenamiento

40



HEAPSORT: inicio

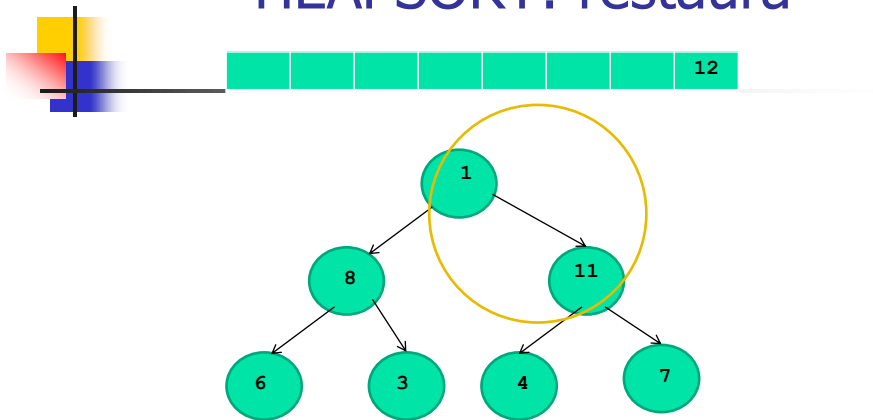


Estructura de Datos

Ordenamiento

41

HEAPSORT: restaura

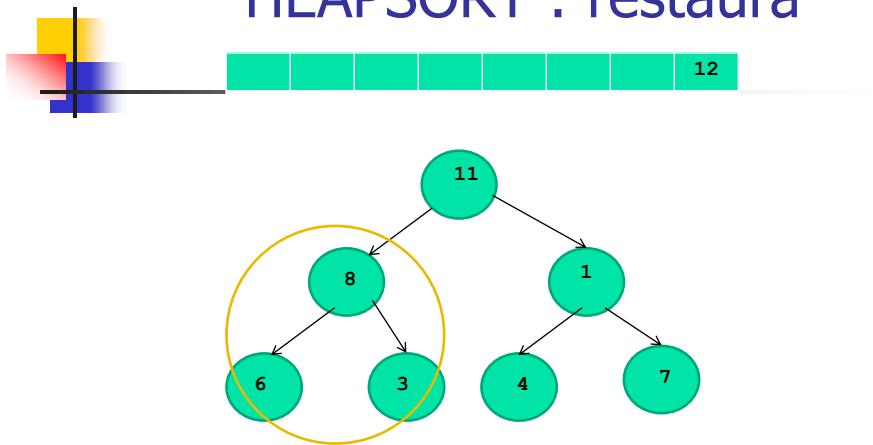


Estructura de Datos

Ordenamiento

42

HEAPSORT : restaura

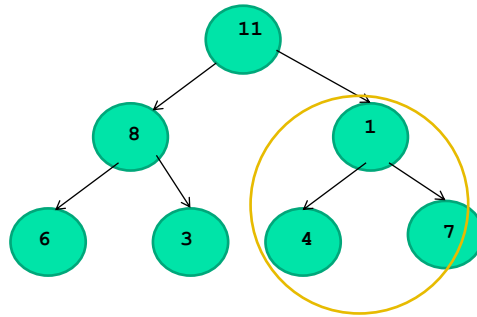


Estructura de Datos

Ordenamiento

43

HEAPSORT : restaura

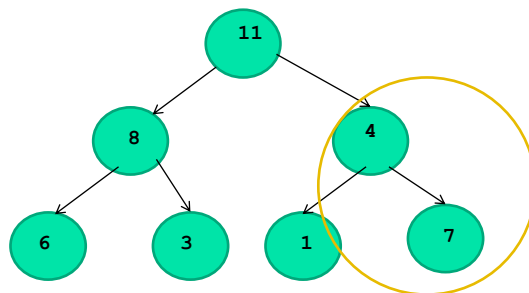


Estructura de Datos

Ordenamiento

44

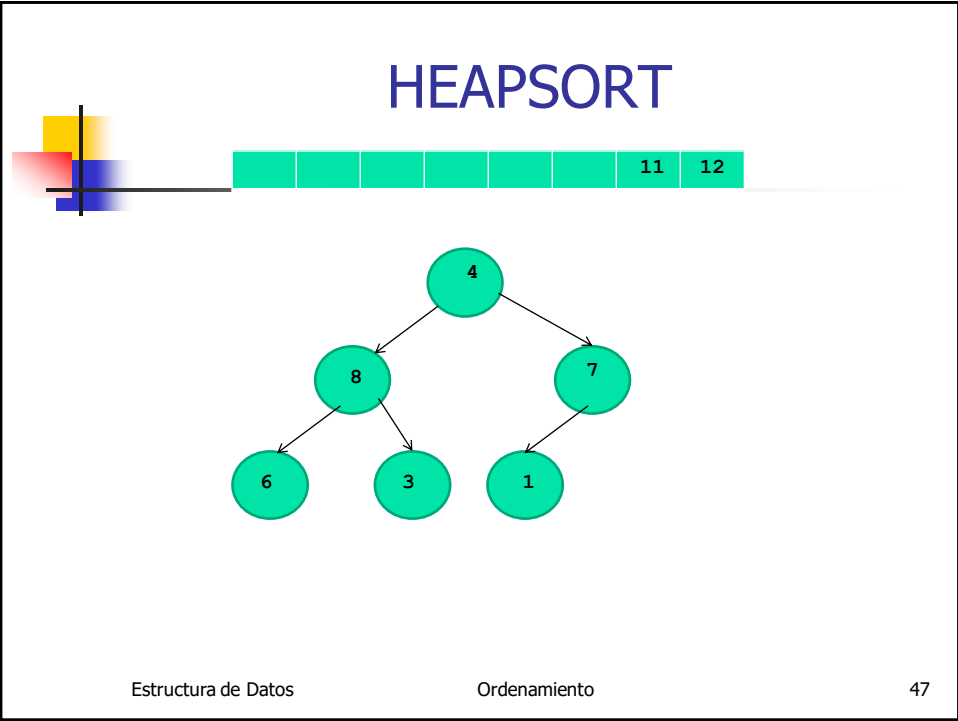
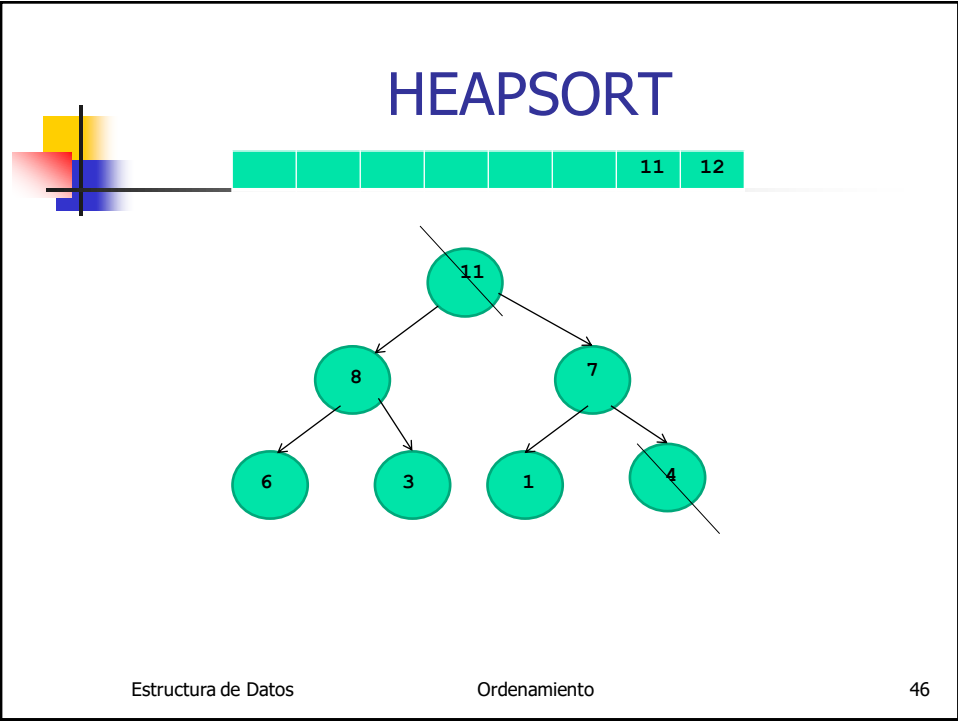
HEAPSORT : restaura



Estructura de Datos

Ordenamiento

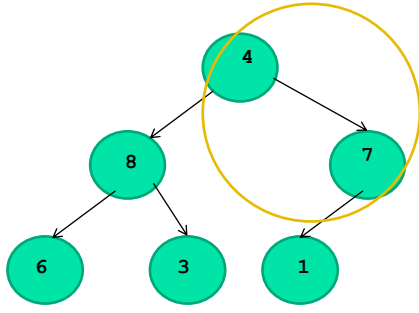
45





HEAPSORT : restaura

						11	12
--	--	--	--	--	--	----	----



Estructura de Datos

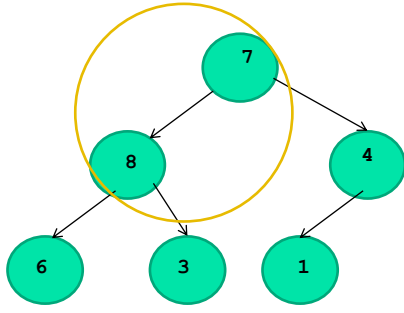
Ordenamiento

48



HEAPSORT : restaura

						11	12
--	--	--	--	--	--	----	----



Estructura de Datos

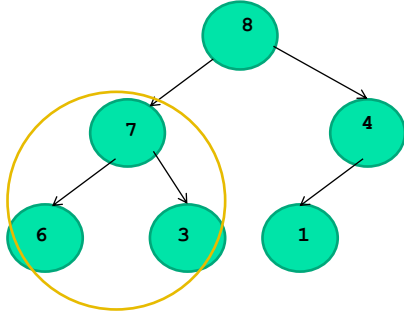
Ordenamiento

49



HEAPSORT : restaura

						11	12
--	--	--	--	--	--	----	----



Estructura de Datos

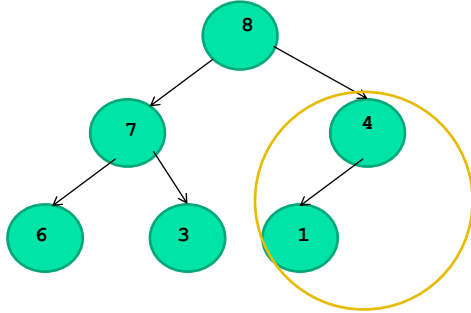
Ordenamiento

50



HEAPSORT : restaura

						11	12
--	--	--	--	--	--	----	----



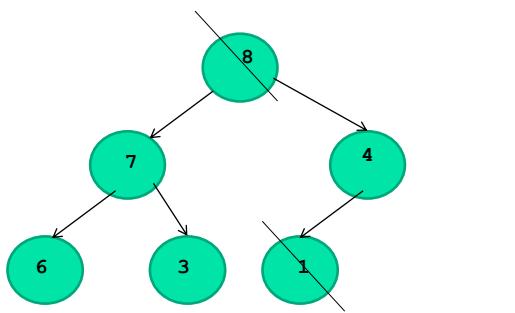
Estructura de Datos

Ordenamiento

51



HEAPSORT



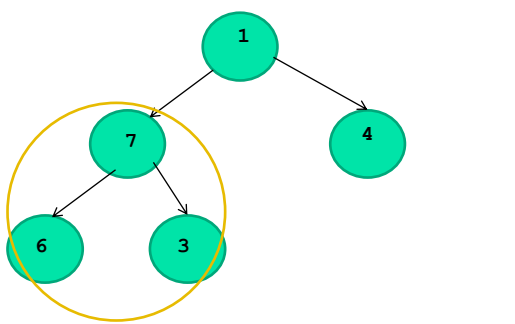
Estructura de Datos

Ordenamiento

52



HEAPSORT : restaura

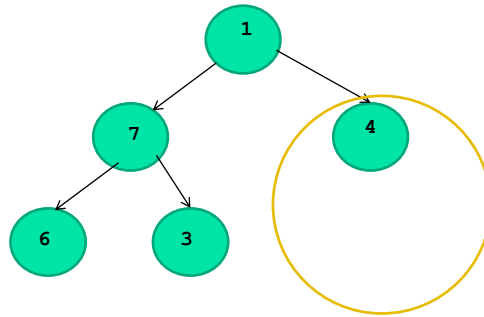


Estructura de Datos

Ordenamiento

53

HEAPSORT : restaura

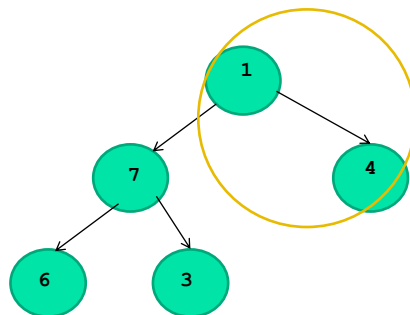


Estructura de Datos

Ordenamiento

54

HEAPSORT : restaura

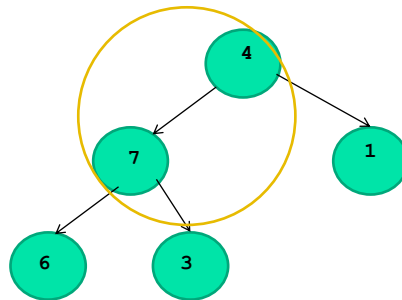


Estructura de Datos

Ordenamiento

55

HEAPSORT : restaura

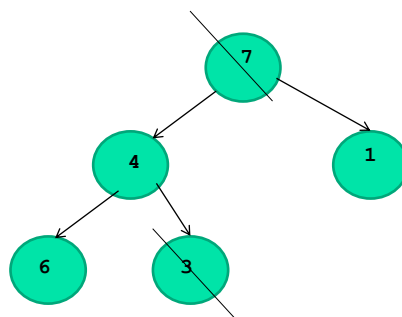


Estructura de Datos

Ordenamiento

56

HEAPSORT

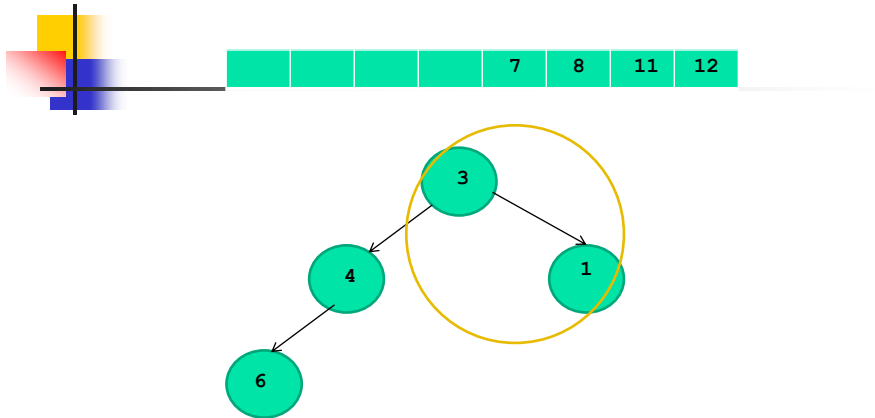


Estructura de Datos

Ordenamiento

57

HEAPSORT : restaura

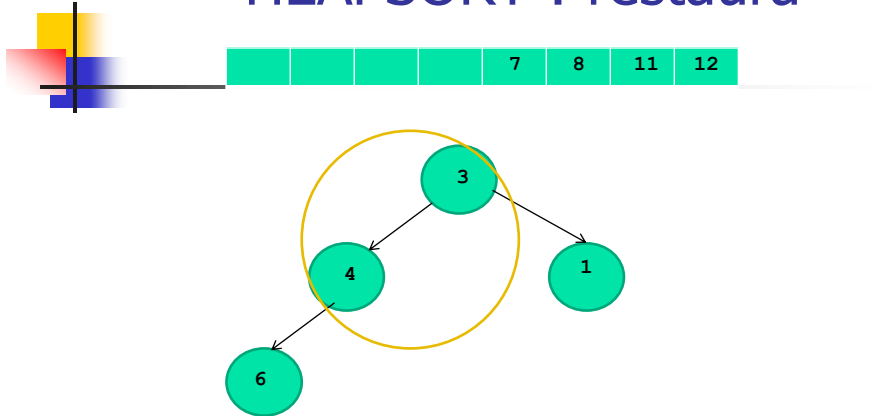


Estructura de Datos

Ordenamiento

58

HEAPSORT : restaura

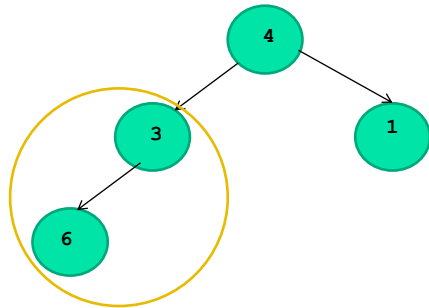


Estructura de Datos

Ordenamiento

59

HEAPSORT : restaura

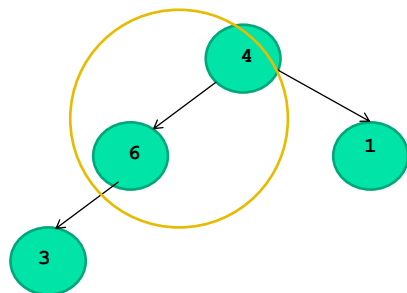
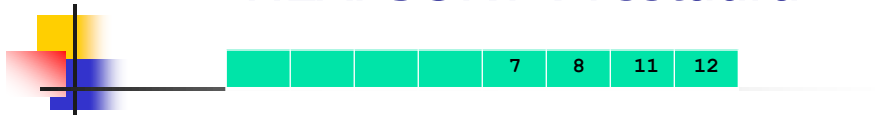


Estructura de Datos

Ordenamiento

60

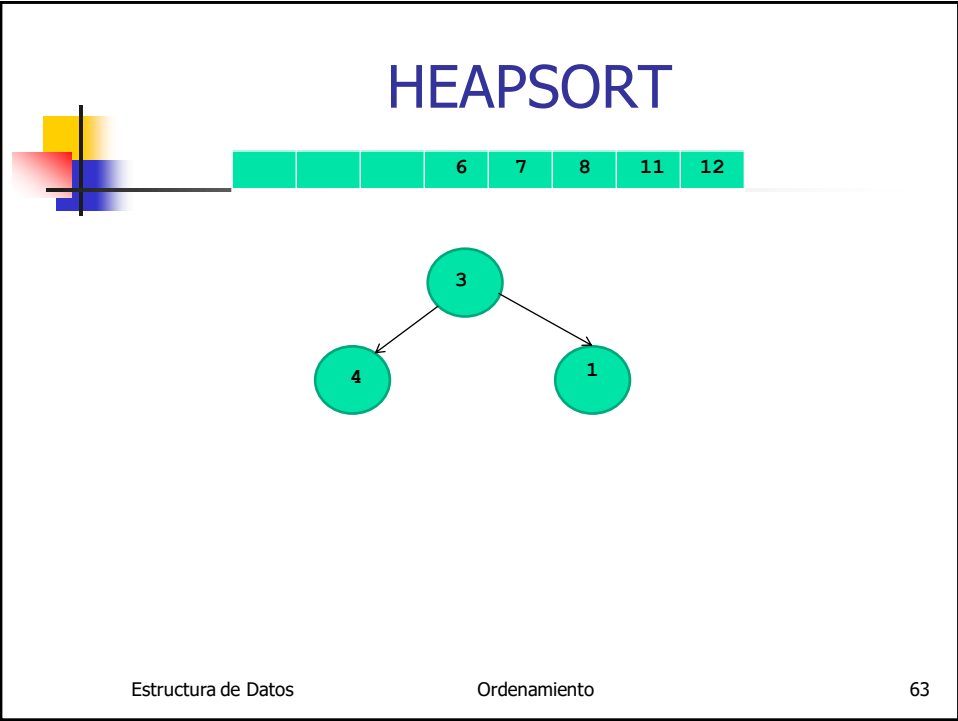
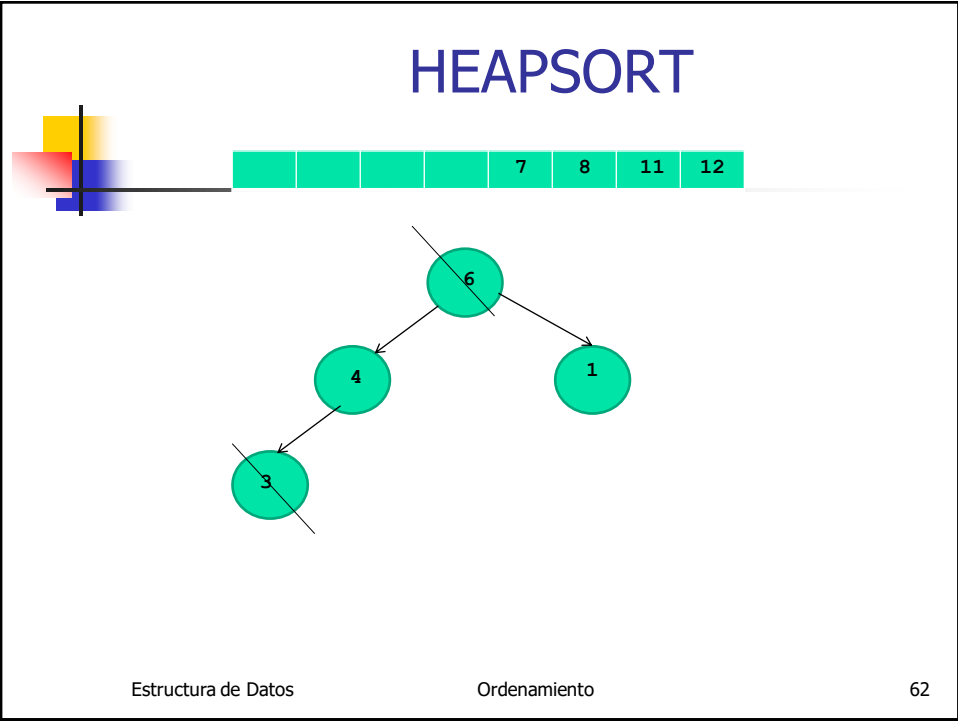
HEAPSORT : restaura



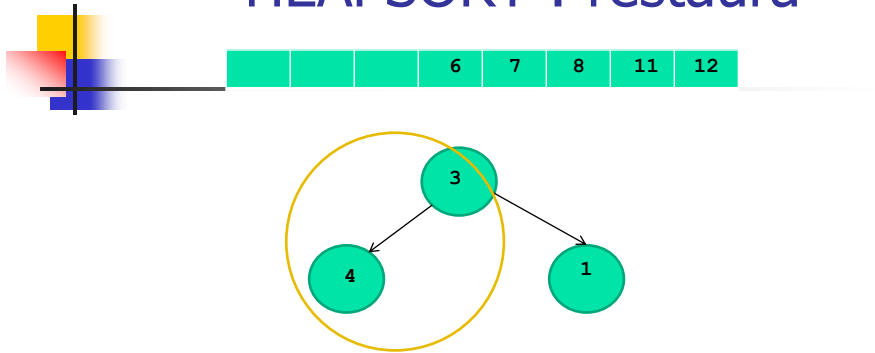
Estructura de Datos

Ordenamiento

61



HEAPSORT : restaura

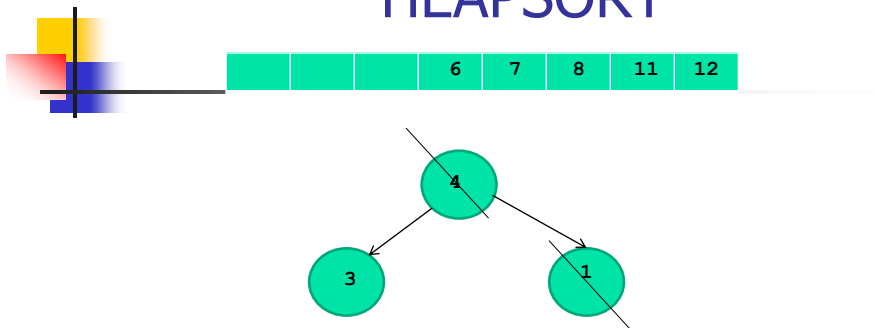


Estructura de Datos

Ordenamiento

64

HEAPSORT

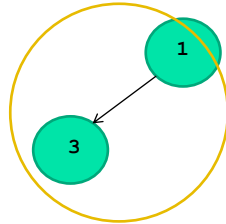


Estructura de Datos

Ordenamiento

65

HEAPSORT : restaura

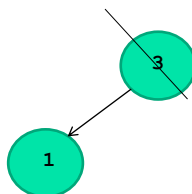
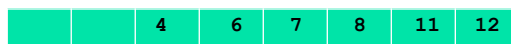


Estructura de Datos

Ordenamiento

66

HEAPSORT



Estructura de Datos

Ordenamiento

67



ARRAY ORDENADO

Heapsort

Análisis de complejidad:

- Se considera dos partes.
 1. Construye inicialmente el montículo a partir de los elementos a ordenar
 2. Por cada iteración recorre un arreglo más pequeño, colocando el elemento raíz en su posición correcta dentro del montículo.
- Peor caso
 - $O(n)$, cuando recorre siempre la máxima profundidad al empujar a cada elemento (arreglo ordenado de forma creciente).
- Mejor caso
 - $O(1)$, cuando la profundidad a la que hay que empujar cada elemento es cero (p.e. elementos iguales).
- Casos mejor y peor del resto del algoritmo heapSort.
 - Peor caso, **$O(n \log n)$** , cuando el elemento es menor que todos los demás
 - Mejor caso, $O(n)$, cuando las raíces no descienden, por ser mayores o iguales que el resto de los elementos del montículo.

Quicksort

- Es el método de ordenación mas utilizado y mas rápido conocido en la práctica.
- Desarrollado en 1960 por Charles Hoare.
- Diseño basado en la técnica "Divide y vencerás".
- Pasos:
 - Dividir: elegir un elemento del arreglo (elemento de división o pivote), particionar el arreglo en dos sub-arreglos no vacíos, tal que cada elemento de un sub-arreglo sea menor o igual a los elementos del otro sub-arreglo.
 - Vencerás: los dos arreglos son ordenados llamando recursivamente a quicksort.
 - Combinar: Como cada arreglo ya está ordenado, no se requiere trabajo adicional.
 - Realizar esto de forma recursiva para cada subvector mientras éstas tengan un largo mayor que 1.
- A continuación se muestra el algoritmo de este método.

Quicksort

```
public static void quickSort(int[] vec) {
    quickSort(vec, 0, vec.length-1);
}

private static void quickSort(int[] vec, int
first, int last) {
    int center=(first + last) / 2;
    if(first<last) {
        center=division(vec, first, last, center);
        quickSort(vec, first, center);
        if(center == first)
            quickSort(vec, center + 1, last);
        else
            quickSort(vec, center, last);
    }
}
```

```
private static int division(int[] vec, int first,
int last, int center) {
    int left, right, data;
    data=vec[center];
    left=first;
    right=last;
    do {
        while(left<right && vec[left]<data)
            left++;
        while(left<right && vec[right]>data)
            right--;
        if(left<right) {
            interChange(vec, left, right);
            left++;
            right--;
        }
    }
    while(left<right);
    if(left<right) {
        int pos=right;
        right=left;
        left=pos;
    }
    return left;
}
```

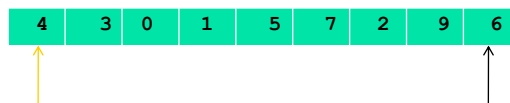
Quicksort

- Pasos:
 1. Seleccionar pivote
 2. Dividir por el pivote
 3. Ordenar los elementos $\leq$ que el pivote (izquierda)
 4. Ordenar los elementos $\geq$ que el pivote (derecha)
- El punto crítico están en los pasos 1 y 2. Existen varios métodos para esta operación:
 - 1er. Método: elegir el primer elemento del arreglo
 - Problema: no se puede colocar nada antes del arreglo
 - Buscar el lugar correcto en el arreglo del pivote
 - Ejemplo:
 - Vector: 7 2 12 9 15 3 10 1 Pivote=1 **v[1] es 7**
 - Quedaría: 2 3 1 **7** 12 9 15 10
 - Continuar con el proceso: escoger pivote y dividir
 - Peor caso
 - Cuando el pivote es igual al menor o mayor valor del arreglo

Quicksort

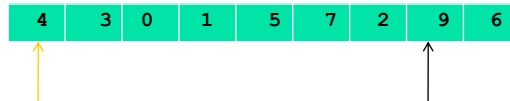
- 2do. Método: elegir el elemento medio del arreglo:
 - $(\text{primero} + \text{ultimo}) / 2$
 - Ejemplo:
 - Vector: 7 2 12 9 15 3 10 1 Pivote=4 **v[4] es 9**
 - Quedaría: 7 2 3 1 **9** 15 10
 - Continuar con el proceso: escoger pivote y dividir

QUICKSORT: ejemplo





QUICKSORT



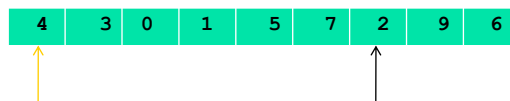
Estructura de Datos

Ordenamiento

76



QUICKSORT



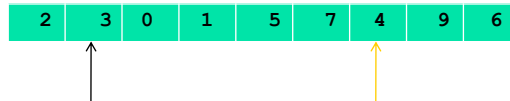
Estructura de Datos

Ordenamiento

77



QUICKSORT



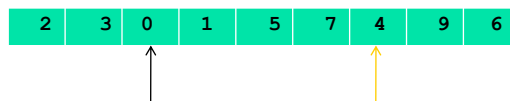
Estructura de Datos

Ordenamiento

78



QUICKSORT



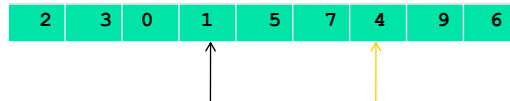
Estructura de Datos

Ordenamiento

79



QUICKSORT



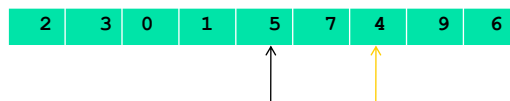
Estructura de Datos

Ordenamiento

80



QUICKSORT



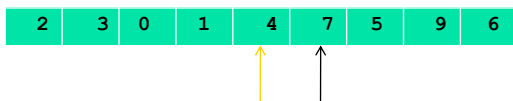
Estructura de Datos

Ordenamiento

81



QUICKSORT



Estructura de Datos

Ordenamiento

82



QUICKSORT



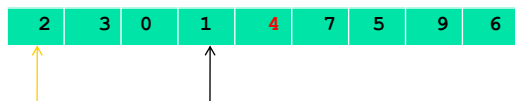
Estructura de Datos

Ordenamiento

83



QUICKSORT



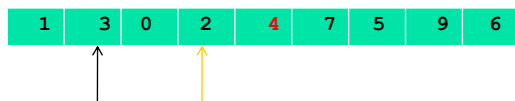
Estructura de Datos

Ordenamiento

84



QUICKSORT



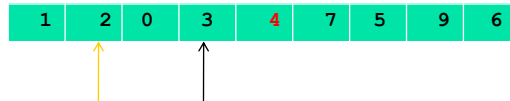
Estructura de Datos

Ordenamiento

85



QUICKSORT



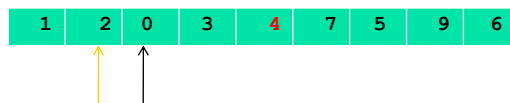
Estructura de Datos

Ordenamiento

86



QUICKSORT



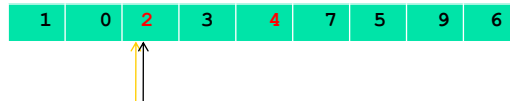
Estructura de Datos

Ordenamiento

87



QUICKSORT



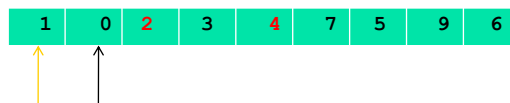
Estructura de Datos

Ordenamiento

88



QUICKSORT



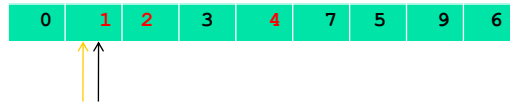
Estructura de Datos

Ordenamiento

89



QUICKSORT



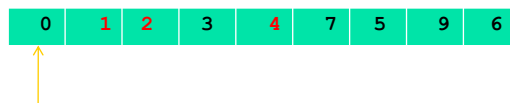
Estructura de Datos

Ordenamiento

90



QUICKSORT



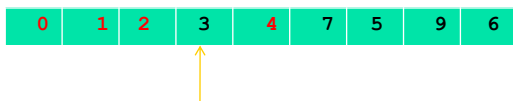
Estructura de Datos

Ordenamiento

91



QUICKSORT



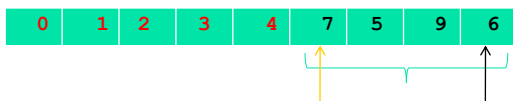
Estructura de Datos

Ordenamiento

92



QUICKSORT



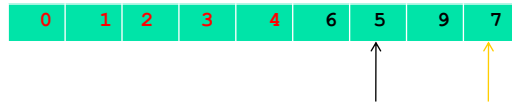
Estructura de Datos

Ordenamiento

93



QUICKSORT



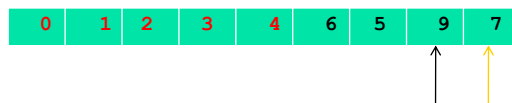
Estructura de Datos

Ordenamiento

94



QUICKSORT



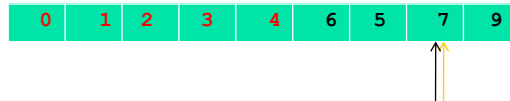
Estructura de Datos

Ordenamiento

95



QUICKSORT



Estructura de Datos

Ordenamiento

96



QUICKSORT



Estructura de Datos

Ordenamiento

97



QUICKSORT



Estructura de Datos

Ordenamiento

98



QUICKSORT



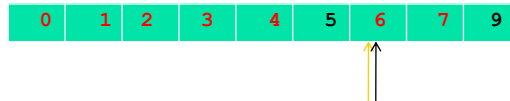
Estructura de Datos

Ordenamiento

99



QUICKSORT



Estructura de Datos

Ordenamiento

100



QUICKSORT



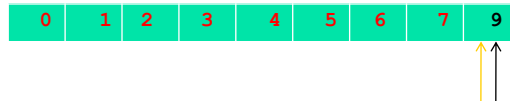
Estructura de Datos

Ordenamiento

101



QUICKSORT



Estructura de Datos

Ordenamiento

102



QUICKSORT



Estructura de Datos

Ordenamiento

103



Quicksort

Análisis de Complejidad:

- De que se supone la eficiencia del algoritmo?
 - depende de la posición en la que termine el pivote elegido.
- En el mejor caso, el pivote termina en el centro de la lista, dividiéndola en dos sublistas de igual tamaño. En este caso, el orden de complejidad del algoritmo es $O(n \cdot \log n)$.
- En el peor caso, el pivote termina en un extremo de la lista. El orden de complejidad del algoritmo es entonces de $O(n^2)$.
- El peor caso dependerá de 2 factores:
 - Array ordenadas, o casi ordenadas.
 - Elección del pivote, si por ejemplo el algoritmo implementado toma como pivote siempre el primer elemento del array, y el array que le pasamos esta ordenado, siempre va a generar a su izquierda un array vacío, lo que es ineficiente.
- En el caso promedio, el orden es $O(n \cdot \log n)$.



Shellsort

- Creado por Donald Shell en el año 1959.
- Es el más eficiente de los del tipo $O(n^2)$. Pero es el más complicado de los algoritmos de este tipo.
- Mejora del método de inserción directa y burbuja
- Puede ser lenta porque intercambia elementos adyacentes
- El Shell sort es una generalización del ordenamiento por inserción, teniendo en cuenta dos observaciones:
 - El ordenamiento por inserción es eficiente si la entrada está "casi ordenada".
 - El ordenamiento por inserción es ineficiente, en general, porque mueve los valores sólo una posición cada vez.
- Compara elementos separados por un espacio de varias posiciones.
- Esto permite que un elemento haga "pasos más grandes" hacia su posición esperada. Los pasos sobre los datos se hacen con tamaños de espacio cada vez más pequeños.
- El último paso del Shellsort es un simple ordenamiento por inserción, pero para entonces, ya está garantizado que los datos del vector están casi ordenados.
- A continuación se muestra el algoritmo de este método.



Shellsort

```
public static void shellSort(int[] vec) {
    int interval=vec.length / 2;
    while(interval>0) {
        for(int i=interval; i<vec.length; i++) {
            int j=i - interval;
            while(j>=0) {
                if(vec[j]<=vec[j + interval])
                    j=0;
                else
                    interChange(vec, j, j + interval);
                j-=interval;
            }
            interval/=2;
        }
    }
}
```



Shellsort

- Pasos:

- Calcular la distancia o salto N, inicialmente será la mitad de los elementos ($N/2$).
- Comparar
 - Si el valor comparado es menor que su predecesor, deberá realizarse un intercambio de valores, en caso contrario quedan en la misma posición
- Se avanza una posición y se compara con el valor ubicado en la distancia N
- Si al realizar un intercambio, a la izquierda del elemento tiene otro a la misma distancia, deberá compararse con este también.
- Una vez terminado el recorrido, se vuelve a calcular la nueva distancia N, haciendo una división por 2 del valor distancia actual.
- Con el nuevo valor distancia. se repite el procedimiento de comparar un elemento con su predecesor.
- Si la distancia es cero, fin ordenamiento.

Shellsort

8	5	3	9	1	4	7
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Formula: $K=N/2$

$K=7/2 = 3$

PRIMERA PASADA: $K=3$

Estructura de Datos

Ordenamiento

108

Shellsort

8	5	3	9	1	4	7
↑			↑			

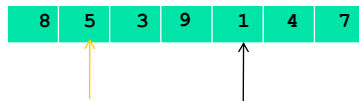
Estructura de Datos

Ordenamiento

109



Shellsort



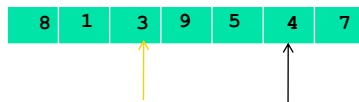
Estructura de Datos

Ordenamiento

110



Shellsort

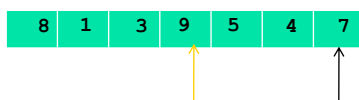


Estructura de Datos

Ordenamiento

111

Shellsort

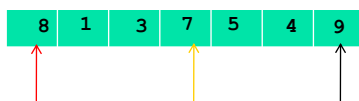


Estructura de Datos

Ordenamiento

112

Shellsort

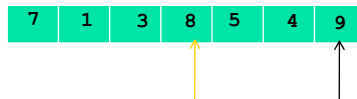


Estructura de Datos

Ordenamiento

113

Shellsort



FIN PRIMERA PASADA: $K=3$

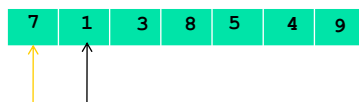
Shellsort



$$K=3/2 = 1.5 = 1$$

SEGUNDA PASADA: $K=1$

Shellsort

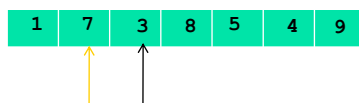


Estructura de Datos

Ordenamiento

116

Shellsort



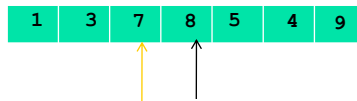
Estructura de Datos

Ordenamiento

117



Shellsort



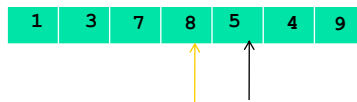
Estructura de Datos

Ordenamiento

118



Shellsort



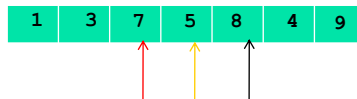
Estructura de Datos

Ordenamiento

119



Shellsort



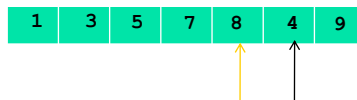
Estructura de Datos

Ordenamiento

120



Shellsort

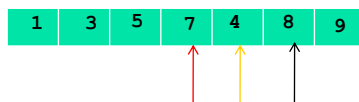


Estructura de Datos

Ordenamiento

121

Shellsort

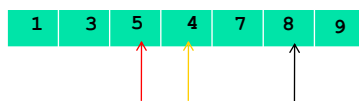


Estructura de Datos

Ordenamiento

122

Shellsort

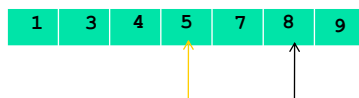


Estructura de Datos

Ordenamiento

123

Shellsort

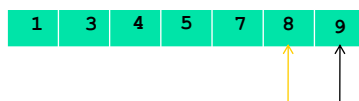


Estructura de Datos

Ordenamiento

124

Shellsort



ARRAY ORDENADO

Estructura de Datos

Ordenamiento

125



Shellsort

- La potencia de este método consiste en conseguir un número de intercambios mucho menor que con la inserción clásica.
- Análisis de complejidad:
 - Difiere al resto de los métodos
 - Complejidad difícil calcular y depende de la secuencia de incrementos.
 - Dependiendo en la elección de la secuencia de saltos, Shellsort a probado tener un tiempo de ejecución en el peor caso igual a $O(n^2)$, $O(n^{3/2})$, $O(n^{4/3})$ o $O(n \log_2 n)$ o posiblemente mejores tiempos de ejecución aún no probados.
 - En 1969 Ronald Pratt descubrió que el tiempo de ejecución del algoritmo es del orden de $O(n \log^2 n)$.



Shakesort

- El método de la sacudida es una variación del método de la burbuja.
- Idea básica:
 - Mezclar las 2 formas en que se pueden realizar el método de la burbuja.
- Cada iteración se realizan dos etapas:
 - 1ra. etapa (de derecha a izquierda)
 - se trasladan los elementos más pequeños hacia la parte izquierda del vector, almacenando en una variable la posición del último elemento intercambiado.
 - 2da. etapa (de izquierda a derecha)
 - se trasladan los elementos más grandes hacia la parte derecha del vector, almacenando en otra variable la posición del último elemento intercambiado.
- Los nuevos barridos se hacen entre los límites marcados por ambas variables.
- El algoritmo termina:
 - Cuando en una etapa no se producen intercambios, o...
 - Cuando el contenido de la variable "izquierda" ha sobrepasado a la variable "derecha".

Shakesort

- A continuación se muestra el algoritmo de este método.

```
public static void shakerSort(int[] vec) {
    int left=1;
    int right=vec.length - 1;
    int aux=vec.length - 1;
    do {
        for(int i=right; i>=left; i--)
            if(vec[i - 1]>vec[i]) {
                interChange(vec, i - 1, i);
                aux=i;
            }
        left=aux + 1;
        for(int i=left; i<=right; i++)
            if(vec[i - 1]>vec[i]) {
                interChange(vec, i - 1, i);
                aux=i;
            }
        right=aux - 1;
    }
    while(left<right);
}
```

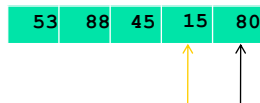
SHAKESORT

53	88	45	15	80
----	----	----	----	----

ARRAY ORIGINAL



SHAKESORT



1ER. PASO:
DERECHA A IZQUIERDA

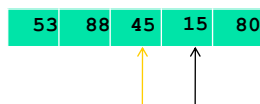
Estructura de Datos

Ordenamiento

130



SHAKESORT



DERECHA A IZQUIERDA

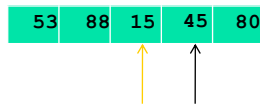
Estructura de Datos

Ordenamiento

131



SHAKESORT



DERECHA A IZQUIERDA

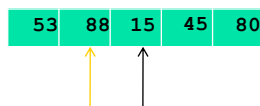
Estructura de Datos

Ordenamiento

132



SHAKESORT



DERECHA A IZQUIERDA

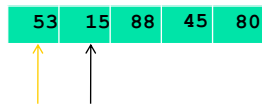
Estructura de Datos

Ordenamiento

133



SHAKESORT



DERECHA A IZQUIERDA

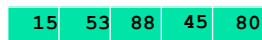
Estructura de Datos

Ordenamiento

134



SHAKESORT



FIN 1ER. PASO
DERECHA A IZQUIERDA

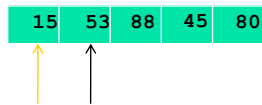
Estructura de Datos

Ordenamiento

135



SHAKESORT



1ER. PASO
DE IZQUIERDA A DERECHA

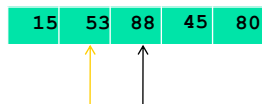
Estructura de Datos

Ordenamiento

136



SHAKESORT



DE IZQUIERDA A DERECHA

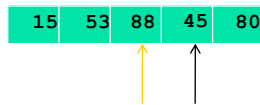
Estructura de Datos

Ordenamiento

137



SHAKESORT



DE IZQUIERDA A DERECHA

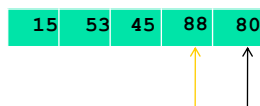
Estructura de Datos

Ordenamiento

138



SHAKESORT



DE IZQUIERDA A DERECHA

Estructura de Datos

Ordenamiento

139



SHAKESORT

15	53	45	80	88
----	----	----	----	----

FIN IZQUIERDA A DERECHA

Estructura de Datos

Ordenamiento

140



SHAKESORT

15	53	45	80	88
----	----	----	----	----



INICIO 2DO PASO
DERECHA A IZQUIERA

Estructura de Datos

Ordenamiento

141



SHAKESORT



2DO PASO
DERECHA A IZQUIERA

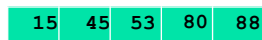
Estructura de Datos

Ordenamiento

142



SHAKESORT



FIN 2DO PASO
DERECHA A IZQUIERDA

Estructura de Datos

Ordenamiento

143



SHAKESORT



INICIO 2DO PASO
IZQUIERDA A DERECHA

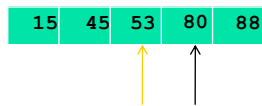
Estructura de Datos

Ordenamiento

144



SHAKESORT



2DO PASO
IZQUIERDA A DERECHA

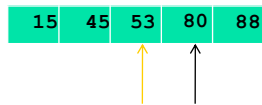
Estructura de Datos

Ordenamiento

145



SHAKESORT



FIN 2DO PASO
IZQUIERDA A DERECHA

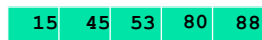
Estructura de Datos

Ordenamiento

146



SHAKESORT



ARRAY ORDENADO

Estructura de Datos

Ordenamiento

147



Shakesort

Análisis de complejidad:

- **La complejidad del método de la sacudida es $O(n^2)$ en el peor caso.**
- El bucle más externo se ejecutará, como máximo, n veces y en el interior del mismo hay dos bucles con complejidad $O(n)$.
- **Sin embargo, el método de la sacudida se aproxima a $O(n)$ para los casos intermedios.**
- El análisis es muy complejo.
- Se tiene en cuenta 3 factores: comparaciones, intercambios y las pasadas que se realizan.
- Otro punto crítico: la operación de intercambio es más complicada y costosa que la comparación.
- Es conveniente cuando el arreglo está parcialmente ordenado.
- Si el arreglo está desordenado es peor que otros métodos como inserción y selección.



Otros algoritmos avanzados de ordenación

RadixSort, BinSort, y Counting Sort

Algoritmos avanzados de ordenación



- Los algoritmos vistos hasta ahora se basan en la ordenación de arreglos de números enteros, sin ningún tipo de restricción.
- A continuación veremos cómo pueden encontrarse algoritmos de orden $O(n)$ cuando dispongamos de información adicional sobre los valores a ordenar.
- Además, como lograr ordenar sin la necesidad de comparaciones e intercambios.

Ordenamiento no Comparativos

- Ordenan usando información adicional
- No se puede aplicar siempre
- Solo cuando se posee esa información adicional



Radixsort: ordenación por residuos



- Algoritmo de ordenamiento que ordena valores compuestos por secuencias de letras o dígitos que admiten un orden (lexicográfico), p.e. palabras, números, o fechas.
- Consiste en definir k colas (0 a k-1) siendo k los posibles valores que puede tomar cada uno de los dígitos que la componen.
- Proceso:
 - Distribuir los elementos en las colas en función del dígito i.
 - Extraer ordenada y consecutivamente los elementos de las colas, introduciéndolos de nuevo en el arreglo.

Estructura de Datos

Ordenamiento

152



Radixsort: ordenación por residuos



- Ejemplo: secuencia de número naturales en base 10.
- Arreglo: [0, 1, 81, 64, 23, 27, 4, 25, 36, 16, 9, 49]
- Como es en base 10 (decimal), se necesitan 10 colas
- 1ra. Pasada:
 - introducimos los elementos en las colas de acuerdo a su dígito menos significativo.

	81			4		16			49
0	1		23	64	25	36	27		9
0	1	2	3	4	5	6	7	8	9

- Se obtiene el siguiente arreglo:
[0, 1, 81, 23, 64, 4, 25, 36, 16, 27, 9, 49]

Estructura de Datos

Ordenamiento

153



Radixsort: ordenación por residuos



- 2da. Pasada:
 - Se introduce a la cola de acuerdo al **segundo dígito** menos significativo.

9									
4		27							
1		25							
0	16	23	36	49		64		81	
0	1	2	3	4	5	6	7	8	9

- Se obtiene el siguiente arreglo:
[0, 1, 4, 9, 16, 23, 25, 27, 36, 49, 64, 81]
- Termina el proceso porque el máximo de dígitos de los números a ordenar era de dos.
- La implementación de este método es el siguiente:

Estructura de Datos

Ordenamiento

154



Radixsort: ordenación por residuos



```
//Parte del código de este algoritmo es similar al de binSort.  
//Por lo tanto, sería óptimo que se reutilice código.  
public static void radixSort(int vec[]) {  
    for(int d=1; d<=/*cantidad de dígitos del máximo elemento en el arreglo*/; d++) {  
        //crea la urna  
        Queue[] urn=new Queue[10];  
        for(int i=0; i<urn.length; i++)  
            urn[i]=new Queue();  
        //guarda cada elemento del arreglo en su correspondiente urna  
        for(int i=0; i<vec.length; i++)  
            urn[getKey(vec[i], d)].insert(new Integer(vec[i]));  
        //recupera cada elemento de las distintas urnas y lo guarda en el arreglo  
        int j=0;  
        for(int i=0; i<urn.length; i++)  
            while(!urn[i].isEmpty()) {  
                vec[j]=((Integer)urn[i].delete()).intValue();  
                j++;  
            }  
    }  
}
```

Estructura de Datos

Ordenamiento

155



Bucket sort (Bin sort)

- Ordenar elementos que pueden no tener un rango enumerable
- Debe ser conocida la distribución de los datos
- Los datos deben ser uniformemente distribuidos



Bin sort: ejemplo

- Ejemplo: ordenar promedio de goles por partido de la selección
- Pasos:
 - Aplicar un criterio de clasificación
 - 1er. paso: Separar por baldes
 - Se asigna elemento a elemento un balde que corresponde a cada promedio de goles.
 - 2do. paso: ordenar cada balde
 - Asumiendo que la distribución es uniforme en cada balde, luego se podría ordenar con un algoritmo de ordenamiento cualquiera.
 - Si no se posee información sobre la distribución, un balde podría tener muchos más elementos que el resto.
 - 3er. paso: Unir resultados
 - Se unen balde por balde (en orden) juntando los resultados.



Bin sort: ejemplo

Jugador	Promedio
Enzo Fernandez	0.11
Mac Allister	0.14
Tagliafico	0.04
Messi	0.91
Julian Alvarez	0.39
Lautaro Martinez	0.40
Di Maria	0.23
Otamendi	0
De Paul	0.11



Bin sort: ejemplo

- Se define criterio
 - Intervalos: $\{<0.03, <0.08, <0.15, <0.25, \text{el resto}\}$
- 1er. Paso: separar en intervalos no uniformes
 - $\{\text{Otamendi}\}$
 - $\{\text{Tagliafico}\}$
 - $\{\text{Mac Allister, De Paul, Enzo Fernandez}\}$
 - $\{\text{Di Maria}\}$
 - $\{\text{Messi, Julian Alvarez, Lautaro Martinez}\}$



Bin sort: ejemplo

- 2do. Paso: ordenar cada balde
 - {Otamendi}
 - {Tagliafico}
 - {De Paul , Enzo Fernandez, Mac Allister}
 - {Di Maria}
 - {Julian Alvarez, Lautaro Martinez, Messi}
- 3er. Paso: unir todos los baldes
 - {Otamendi, Tagliafico, De Paul, Enzo Fernandez, Mac Allister, Di Maria, Julian Alvarez, Lautaro Martinez, Messi}
- Ya esta ordenado por promedio de goles por partido



Counting sort

- Fue propuesto por Harold H. Seward en 1.954.
- La idea principal es contar el número de ocurrencias de cada elemento en la lista de entrada y utilizar esta información para construir una lista ordenada.
- Es un algoritmo que sólo fue diseñado para ordenar números enteros sin necesidad de realizar comparaciones



Características

- La complejidad computacional es de $O(n+k)$
 - siendo n el número de elementos a ordenar y k el tamaño del vector auxiliar (máximo - mínimo).
- No existe un mejor y peor caso, todos los casos se tratan iguales.
- Esta técnica de clasificación es efectiva cuando la diferencia entre las diferentes claves no es tan grande, de lo contrario, puede aumentar la complejidad del espacio (tienen un rango limitado).



Ventajas y desventajas

- Ventajas
 - Eficiencia en tiempo lineal
 - complejidad lineal $O(n + k)$, donde " n " es el número de elementos en la lista y " k " es el rango de valores en la lista.
 - Estabilidad
 - No requiere comparaciones entre elementos
 - Adaptable para rangos pequeños de valores
- Desventajas
 - Requiere espacio adicional
 - requiere un arreglo de tamaño igual al rango de valores en la lista, especialmente cuando el rango de valores es grande.
 - No adecuado para rangos grandes de valores
 - No es adecuado para datos no enteros: cadenas de texto o datos de punto flotante.



Pasos

- Verificar el intervalo dentro del que están los datos a ordenar (valores mínimo y máximo).
- Crear un vector de números enteros con tantos elementos como valores haya en el intervalo [mínimo, máximo].
- Contar el número de apariciones (frecuencias) de cada elemento y agregar en el índice que corresponda del vector auxiliar.
- El vector auxiliar contiene las posiciones que ocuparán los valores en el arreglo final.
- Actualizar el vector resultante, agregando consecutivamente cada elemento contenido en el vector auxiliar, teniendo en cuenta su índice, y el valor contenido en dicha posición, indica su posición final en el vector. Recordar decrementar el valor que corresponde con el índice en el arreglo auxiliar.
- Por último, basta con recorrer este vector para tener todos los elementos ordenados.

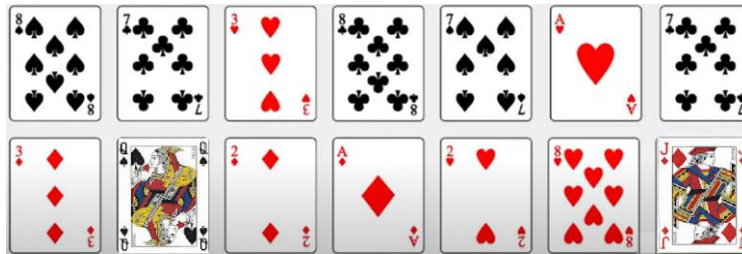


Algoritmo

```
public int[] countingSort(int[] Vector) {  
    int n = Vector.length;  
    int mayor = Vector[0];  
    int menor = Vector[0];  
    for (int i = 1; i < n; i++) {  
        if (Vector[i] > mayor)  
            mayor = Vector[i];  
        if (Vector[i] < menor)  
            menor = Vector[i];  
    }  
    int rango = mayor - menor + 1;  
    int[] Vcuenta = new int[rango];  
    for (int i = 0; i < n; i++)  
        Vcuenta[Vector[i] - menor]++;  
    for (int i = 1; i < rango; i++)  
        Vcuenta[i] += Vcuenta[i - 1];  
    int j = 0;  
    for (int i = 0; i < rango; i++)  
        while (j < Vcuenta[i])  
            Vector[j++] = i + menor;  
    return Vector;  
}
```

Ejemplo

- Ordenar un arreglo de cartas. No necesariamente están todas las cartas, pueden estar repetidas.
- Ordenar simplemente por numero
- Se sabe que los números van del 1 al 11 (cartas francesas)



Estructura de Datos

Ordenamiento

166

Counting sort

- Los valores son:
 - 8, 7, 3, 8, 7, 1, 7, 3, 12 ("Q"), 2, 1, 2, 8, 11 ("J")
- Paso 1:
 - Crear un arreglo auxiliar cuyo tamaño depende del elemento que tengo el mayor valor + 1.
 - En este caso 12 ($12 + 1 = 13$)

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0

Estructura de Datos

Ordenamiento

167

Counting sort

■ Paso 2:

- Contar el numero de apariciones de cada elemento en el arreglo auxiliar, y agregarlo en el índice que corresponda en el vector.
- Ejemplo: 8 se repite 3 veces.
- Los valores son:
 - 8, 7, 3, 8, 7, 1, 7, 3, 12, 2, 1, 2, 8, 11

0	1	2	3	4	5	6	7	8	9	10	11	12
0	2	2	2	0	0	0	3	3	0	0	1	1

Counting sort

■ Paso 3:

- Se recorre el arreglo sumándose los valores de los índices (anterior + actual) al actual.

0	1	2	3	4	5	6	7	8	9	10	11	12
0	2	2	2	0	0	0	3	3	0	0	1	1

+ =

0	1	2	3	4	5	6	7	8	9	10	11	12
0	2	4	6	6	6	6	9	12	12	12	13	14

Counting sort

Paso 4:

- A través del arreglo auxiliar, se sabe que posición ocuparán los valores en el arreglo final. Ejemplo: 8 ocupa la posición 12 en el arreglo resultante.
- Recordar decrementar el valor que corresponde con el índice en el arreglo auxiliar.
- El tamaño del vector resultante, es el resultado de la sumatoria + 1 del vector auxiliar. En este caso $14 + 1 = 15$

0	1	2	3	4	5	6	7	8	9	10	11	12
0	2	4	6	6	6	6	9	12	12	12	13	14

Red arrows indicating decrements: 12 to 11, 11 to 10, 10 to 9, 9 to 8, 8 to 7, 7 to 6, 6 to 5, 5 to 4, 4 to 3, 3 to 2, 2 to 1, 1 to 0.

- Los valores son: 8, 7, 3, 8, 7, 1, 7, 3, 12, 2, 1, 2, 8, 11

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	1	1	2	2	3	3	7	7	7	8	8	8	11	12

Estructura de Datos

Ordenamiento

170

Orden de Magnitud Algoritmos de ordenamiento

Algorithm	Time Complexity		
	Best	Average	Worst
<u>Quicksort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$
<u>Mergesort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
<u>Timsort</u>	$O(n)$	$O(n \log(n))$	$O(n \log(n))$
<u>Heapsort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
<u>Bubble Sort</u>	$O(n)$	$O(n^2)$	$O(n^2)$
<u>Insertion Sort</u>	$O(n)$	$O(n^2)$	$O(n^2)$
<u>Selection Sort</u>	$O(n^2)$	$O(n^2)$	$O(n^2)$
<u>Tree Sort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$
<u>Shell Sort</u>	$O(n \log(n))$	$O(n(\log(n))^2)$	$O(n(\log(n))^2)$
<u>Bucket Sort</u>	$O(n+k)$	$O(n+k)$	$O(n^2)$
<u>Radix Sort</u>	$O(nk)$	$O(nk)$	$O(nk)$
<u>Counting Sort</u>	$O(n+k)$	$O(n+k)$	$O(n+k)$
<u>Cubesort</u>	$O(n)$	$O(n \log(n))$	$O(n \log(n))$

Estructura de Datos

Ordenamiento

171



Consultas...

¿?



Bibliografía

- Weiss, Mark Allen: "Estructuras de datos y algoritmos", Addison-Wesley Iberoamericana, 1995.
- Estructuras de datos y algoritmos: Alfred Aho, J. Hopcroft y J. Ullman, Addison-Wesley, 1988
- Algoritmos+ Estructuras = Programas: Nicklaus Wirth. Prentice Hall, 1989
- Brassard, Gilles; Bratley, Paul: "Algorithmics: theory and practice", Prentice-Hall, 1988.
- Introduction to Algorithms (4th edition) - Cormen, Leiserson, Rivest, Stein
- Data Structures and Algorithm Analysis - Mark Allen Weiss
- Kleinberg, Jon; Tardos, Éva: Algorithm Design, Addison-Wesley, 2005.
- Cormen, Thomas H.; Leiserson Charles E.; Rivest Ronald L; Stein Clifford: "Introduction to Algorithms", The MIT Press, 2009.